

✓ Network Implementation

Task 2.1

Recall that for one-layer networks, we define parameter $\Theta = \vec{w} \in \mathbb{R}^{D+1}$, and for a two-layer network, we define parameters $\Theta = (\mathbf{W}^{(1)} \in \mathbb{R}^{K \times (D+1)}, \vec{w}^{(2)} \in \mathbb{R}^{K+1})$.

- D: The dimension of the input. In this assignment, $D = 1$ since there is only one input.
- K: The number of neurons in the hidden layer of the two-layer network (N_2)

Implement a function that returns the network output for a given input $\vec{x}$, model_type (N_0 , N_1 , or N_2) and parameters Θ . Remember that the input of the function $\vec{x} = (1, x)^T$.

Note:

1. Use the `numpy` to implement the `tanh` function.
2. Use `numpy.concatenate` or `numpy.insert` to prepend h_0 .
3. Make use of `numpy.dot` to compute matrix-vector and vector-vector products.

```
import numpy

def network(x, model_type, Theta):
    """
    Compute the output of a neural network model.

    Args:
        x: Input vector (1, x) including bias.
        model_type: Type of model (0, 1, or 2).
        Theta: Tuple of network parameters (w1, w2).

    Returns:
        y: Network output.
        h: Hidden layer output, or None.
    """

    w1, w2 = Theta # w2 is None if model_type is 0 or 1

    if model_type == 0:
        # One-layer network (Linear Model)
        y = numpy.dot(w1, x)
        return y, None

    elif model_type == 1:
        # One-layer network with tanh activation
        y = numpy.tanh(numpy.dot(w1, x))
        return y, None
```

```

elif model_type == 2:
    # Two-layer network with tanh activation
    a_ = numpy.dot(W1, x)
    h_ = numpy.tanh(a_)
    h = numpy.concatenate(([1], h_)) # Prepend bias to hidden layer outp
    y = numpy.dot(w2, h)
    return y, h

```

✓ Test 1: Sanity Check

We select a specific number of hidden neurons and create the weights accordingly, using all zeros in the first layer and all ones in the second. The test case below ensures that the function from Task 1 actually returns 11 for those weights.

```

# Define test parameters
K_ = 20
D_ = 1
Theta_one_layer = [numpy.ones(D_+1),None]
Theta_two_layer = [numpy.zeros((K_, D_+1)), numpy.ones(K_+1)]
x = numpy.random.rand(D_+1)

# Sanity check for N1
y1, _ = network(x, 0, Theta_one_layer)
assert abs(numpy.sum(x) - y1) < 1e-6
print("N1 test passed.")

# Sanity check for N2
y2, _ = network(x, 1, Theta_one_layer)
assert abs(numpy.tanh(numpy.sum(x)) - y2) < 1e-6
print("N2 test passed.")

# Sanity check for N3
y3, _ = network(x, 2, Theta_two_layer)
assert abs(1.0 - y3) < 1e-6
print("N3 test passed.")

N1 test passed.
N2 test passed.
N3 test passed.

```

✓ Gradient Implementation

Task 2.2: Gradient Computation

Implementation of a function that returns the gradient as defined for a given dataset $X = \{(\vec{x}^{[n]}, t^{[n]})\}$, model_type, parameters Θ , and the λ parameter for weight decay.

Note:

You should make sure that both parts of the gradient are computed for N_2 (since $\Theta = (\mathbf{W}^{(1)}, \vec{w}^{(2)})$ here).

This is a very slow implementation. We will see how to speed this up in the next lecture.

```
def compute_gradient(X, model_type, Theta, lambda_=1.):
    """
    Compute the gradient of the loss function with respect to the weights fo

    Args:
        X: Dataset containing input-target pairs (x, t).
        model_type: Type of model (0, 1, or 2).
        Theta: Network parameters (W1, w2).
        lambda_: Weight decay parameter. Default is 1.0.

    Returns:
        Gradients with respect to W1 and w2. For model_type 1 and 2, w2 is N
    """

    # split parameters for easier handling
    W1, w2 = Theta # w2 is None if model_type is 0 or 1

    # define gradient with respect to both parameters
    dW1 = numpy.zeros_like(W1)
    if w2 is not None:
        dw2 = numpy.zeros_like(w2)
    else:
        dw2 = None

    # iterate over dataset
    for x_sample, t_sample in X:
        # get network output
        y, h = network(x_sample, model_type, Theta)
        err = y - t_sample

        # compute and aggregate the gradient, based on the network type
        if model_type == 0:
            # One-layer network (Linear Model)
            # dL/dW1 = (y - t) * x
            dW1 += err * x_sample

        elif model_type == 1:
            # One-layer network with tanh activation
            # dL/dW1 = (y - t) * (1 - y^2) * x
            dW1 += err * (1 - y**2) * x_sample

        elif model_type == 2:
            # Two-layer network with tanh activation
            # dL/dw2 = (y - t) * h
            dw2 += err * h

    # dL/dW1
```

```

# delta_a = dL/dy * dy/dh_k * dh_k/da_k
# dL/dy = (y - t)
# dy/dh_k = w2_k (for k > 0, as h[0] is bias)
# dh_k/da_k = (1 - h_k^2) where h_k = tanh(a_k)
# dL/dw1_kj = dL/da_k * da_k/dw1_kj = dL/da_k * x_j

# h[1:] contains the tanh outputs of the hidden layer (excluding
# w2[1:] contains the weights connecting hidden layer to output
delta_a = err * w2[1:] * (1 - h[1:]**2)
dw1 += numpy.outer(delta_a, x_sample)

# Add penalty term/weight decay
dw1 += lambda_ * w1
if dw2 is not None:
    dw2 += lambda_ * w2

return dw1, dw2

```

✓ Task 2.3: Gradient Descent

The procedure of gradient descent is the repeated application of two steps.

1. The gradient of loss $\nabla_{\Theta} \mathcal{J}^{L_2}$ is computed based on the current value of the parameters Θ .
2. The weights are updated by moving a small step in the direction of the negative gradient:

$$\Theta = \Theta - \eta \nabla_{\Theta} \mathcal{J}$$

As a stopping criterion, we select the number of training epochs to be 10000.

Implementation of a function that performs gradient descent for a given dataset X , given initial parameters Θ , a given learning rate η , model_type (N_1 , N_2 , or N_3), and λ parameter for weight decay, and returns the optimized parameters Θ^* .

```

def gradient_descent(X, model_type, Theta, eta, lambda_=1., epochs=10000):
    """
    Perform gradient decent for each model type.

    Args:
        X: Dataset containing input-target pairs (x, t).
        model_type: Type of model (0, 1, or 2).
        Theta: Network parameters (w1, w2).
        eta: Learning rate.
        lambda_: Weight decay parameter. Default is 1.0.
        epochs: The number of epochs to train for

    Returns:
        The optimized model parameters
    """

    w1, w2 = Theta

```

```

# perform iterative gradient descent
for _ in range(epochs):
    # compute the gradient
    dw1, dw2 = compute_gradient(X, model_type, (w1, w2), lambda_)

    # update the parameters
    w1 = w1 - eta * dw1
    if w2 is not None:
        w2 = w2 - eta * dw2

# return optimized parameters
return (w1, w2)

```

✓ Datasets

Task 2.4: Data Samples

In total, we will test our gradient descent function with three different datasets. Particularly, we approximate

1. $X_0 : t = \sin(2x)$ for $x \in [-2, 2]$
2. $X_1 : t = e^{-x^2}$ for $x \in [-3, 3]$
3. $X_2 : t = -x^5 - 3x^4 + 11x^3 + 27x^2 - 10x - 32$ for $x \in [-4.5, 3.5]$

Generate dataset X_0 for $N = 45$ samples randomly drawn from range $x \in [-2, 2]$.

Generate dataset X_1 for $N = 15$ samples randomly drawn from range $x \in [-3, 3]$.

Generate dataset X_2 for $N = 250$ samples randomly drawn from range

$x \in [-4.5, 3.5]$. Implement all three datasets as lists of tuples:

$$\{(\vec{x}^{[n]}, t^{[n]}) \mid 1 \leq n \leq N\}$$

```

# Dataset X0: t = sin(2x) for x in [-2,2] with N=45 samples
N0 = 45
min_x0, max_x0 = -2, 2
x_values0 = numpy.random.uniform(min_x0, max_x0, N0)
X0 = []
for x_val in x_values0:
    input_vector = numpy.array([1, x_val])
    t_value = numpy.sin(2 * x_val)
    X0.append((input_vector, t_value))

# Dataset X1: t = e^(-x^2) for x in [-3,3] with N=15 samples
N1 = 15
min_x1, max_x1 = -3, 3
x_values1 = numpy.random.uniform(min_x1, max_x1, N1)
X1 = []
for x_val in x_values1:
    input_vector = numpy.array([1, x_val])
    t_value = numpy.exp(-x_val**2)
    X1.append((input_vector, t_value))

```

```
# Dataset X2:  $t = -x^5 - 3x^4 + 11x^3 + 27x^2 - 10x - 32$  for  $x$  in  $[-4.5, 3.5]$ 
N2 = 250
min_x2, max_x2 = -4.5, 3.5
x_values2 = numpy.random.uniform(min_x2, max_x2, N2)
X2 = []
for x_val in x_values2:
    input_vector = numpy.array([1, x_val])
    t_value = -x_val**5 - 3*x_val**4 + 11*x_val**3 + 27*x_val**2 - 10*x_val
    X2.append((input_vector, t_value))
```

✓ Test 2: Sanity Check

The test case below ensures that the elements of each generated dataset are tuples with two dimensions. It does not test that the values are correct.

```
for i, X in enumerate((X0, X1, X2)):
    assert all(
        isinstance(x, (tuple, list)) and
        len(x) == 2 and
        isinstance(x[0], (tuple, list, numpy.ndarray)) and
        len(x[0]) == 2 and
        isinstance(x[1], (float, numpy.float32, numpy.float64))
        for x in X
    ), f"Test failed for dataset X{i}"

print('Test passed!')
```

Test passed!

✓ Function Approximation

Finally, we want to make use of our gradient descent implementation to approximate our functions. In order to see our success, we want to plot the functions together with the data.

Task 2.5: Define Hidden Neurons

How many hidden neurons will we need for N_2 ? Use the answers from Task 1.2.

```
# Define the number of neurons for each target function based on your discus
K0 = 20
K1 = 20
K2 = 20
```

✓ Task 2.6: Random Parameters

For each of the networks, randomly initialize the parameters $\Theta_0, \Theta_1, \Theta_2 \in [-1, 1]$ for each of the datasets.

For N_2 , use the number of hidden neurons estimated in Task 1.2 and implemented in Task 2.5.

Note:

1. You can use `numpy.random.uniform` to initialize the weights.
2. Make sure that the weight matrices are instantiated in the correct dimensions.
3. Theta should always have two elements. The second element can be `None` for one-layer networks.

D=1

```
# Initialize weights for One-Layer Network (No Activation), once for each da
Thetas_N0 = [
    (numpy.random.uniform(-1, 1, D + 1), None), # For X0
    (numpy.random.uniform(-1, 1, D + 1), None), # For X1
    (numpy.random.uniform(-1, 1, D + 1), None) # For X2
]

# Initialize weights for One-Layer Network (With Activation)
Thetas_N1 = [
    (numpy.random.uniform(-1, 1, D + 1), None), # For X0
    (numpy.random.uniform(-1, 1, D + 1), None), # For X1
    (numpy.random.uniform(-1, 1, D + 1), None) # For X2
]

# Initialize weights for Two-Layer Network
Thetas_N2 = [
    (numpy.random.uniform(-1, 1, (K0, D+1)), numpy.random.uniform(-1, 1, K0+
    (numpy.random.uniform(-1, 1, (K1, D+1)), numpy.random.uniform(-1, 1, K1+
    (numpy.random.uniform(-1, 1, (K2, D+1)), numpy.random.uniform(-1, 1, K2+
]
```

✓ Task 2.7: Run Gradient Descent

For each network, call gradient descent function from Task 2.3 using the datasets X_0, X_1, X_2 , the according created parameters $\Theta_0, \Theta_1, \Theta_2$. Store the resulting optimized weights $\Theta_0^*, \Theta_1^*, \Theta_2^*$.

Based on your chosen learning rates η and weight decay parameter λ , you may need to optimize them for these functions. Do you see any differences? What are the best learning rates that you can find?

WARNING: Depending on the implementation, this might run for several minutes!

Hints:

1. Start with $\eta = 0.1$ and play around with the learning rate improve adaptation.
2. $\eta = 0.1$ might be too large for X_2 .

```
# Call gradient descent function using the datasets and initial weights that
# Choose appropriate learning rates for each function

OptimizedThetas_N0 = []
OptimizedThetas_N1 = []
OptimizedThetas_N2 = []

datasets = [X0, X1, X2]

# N0 (model_type = 0)
# Learning rates for N0 for X0, X1, X2 respectively
etas_N0 = [0.1, 0.1, 0.1]
for i, X_dataset in enumerate(datasets):
    eta = etas_N0[i]
    optimized_theta = gradient_descent(X_dataset, 0, Thetas_N0[i], eta)
    OptimizedThetas_N0.append(optimized_theta)

# N1 (model_type = 1)
# Learning rates for N1 for X0, X1, X2 respectively
etas_N1 = [0.1, 0.1, 0.1]
for i, X_dataset in enumerate(datasets):
    eta = etas_N1[i]
    optimized_theta = gradient_descent(X_dataset, 1, Thetas_N1[i], eta)
    OptimizedThetas_N1.append(optimized_theta)

# N2 (model_type = 2)
# Learning rates for N2 for X0, X1, X2 respectively. X2 often needs a smaller
# Adjusted eta for X2 to a smaller value based on the hint.
etas_N2 = [0.01, 0.01, 0.001]
for i, X_dataset in enumerate(datasets):
    eta = etas_N2[i]
    optimized_theta = gradient_descent(X_dataset, 2, Thetas_N2[i], eta)
    OptimizedThetas_N2.append(optimized_theta)

print("Gradient descent completed for all models and datasets.")
```

```
/tmp/ipykernel_2141/3958262731.py:35: RuntimeWarning: overflow encountered in
  dw1 += err * x_sample
/tmp/ipykernel_2141/3958262731.py:35: RuntimeWarning: invalid value encountered
  dw1 += err * x_sample
/tmp/ipykernel_2141/3926039300.py:25: RuntimeWarning: invalid value encountered
  w1 = w1 - eta * dw1
/tmp/ipykernel_2141/3958262731.py:56: RuntimeWarning: overflow encountered in
  delta_a = err * w2[1:] * (1 - h[1:]**2)
/tmp/ipykernel_2141/3958262731.py:56: RuntimeWarning: invalid value encountered
  delta_a = err * w2[1:] * (1 - h[1:]**2)
Gradient descent completed for all models and datasets.
```

✓ Data and Function Plotting

Task 2.8: Plotting Function

Implement a plotting function that takes a given dataset X , model_type, parameters Θ , and a defined range $R = [\min, \max]$. Each data sample $(\vec{x}^{[n]}, t^{[n]})$ of the dataset is plotted as an "x". In order to plot the function that is approximated by the network, generate sufficient equally-spaced input values $x \in R$, compute the network output y for these inputs, and plot them with a line.

Hints:

1. The dataset X is defined as above, a list of tuples $(\vec{x}, t)$.
2. Each input in the dataset is defined as $\vec{x} = (1, x)^T$.
3. Equidistant points can be obtained via `numpy.arange`.

```
from matplotlib import pyplot
def plot(X, model_type, Theta, R):
    """
    Plots the data and the network output into one plot

    Args:
        X: Dataset containing input-target pairs (x, t).
        model_type: Type of model (0, 1, or 2).
        Theta: Network parameters (w1, w2).
        R: The range for which we plot the model output

    Returns:
        The optimized model parameters
    """
    # first, plot data samples
    x_data = [x_sample[1] for x_sample, _ in X]
    t_data = [t_sample for _, t_sample in X]
    pyplot.plot(x_data, t_data, "rx", label="Data")

    # define equidistant points from min (R[0]) to max (R[1]) to evaluate th
    x_plot = numpy.linspace(R[0], R[1], 200) # Generate 200 equidistant poin
    # compute the network outputs for these values
    y_plot = []
    for val in x_plot:
        input_vec = numpy.array([1, val])
        y_pred, _ = network(input_vec, model_type, Theta)
        y_plot.append(y_pred)
    y_plot = numpy.array(y_plot)

    # plot network approximation
    pyplot.plot(x_plot, y_plot, "k-", label="Network")
    pyplot.legend()
```

✓ Task 2.9: Plot Three Functions for three datasets

For each of the datasets and for each network type including their according optimized parameters, call the plotting function from Task 2.8. Use range $R = [-3, 3]$ for dataset X_0 , range $R = [-4, 4]$ for X_1 , and range $R = [-6, 5]$ for dataset X_2 .

Note that the first element of range R should be the lowest x -location, and the second element of R the highest value for x .

Repeat for three networks.

```
# create one large figure
figure = pyplot.figure(figsize=(15,9))

datasets = [X0, X1, X2]
ranges = {
    0: [-3, 3], # Range for X0
    1: [-4, 4], # Range for X1
    2: [-6, 5] # Range for X2
}

optimized_thetas_all_models = [
    OptimizedThetas_N0,
    OptimizedThetas_N1,
    OptimizedThetas_N2
]

# iterate over the model types
for model_type in range(3):
    # iterate over the datasets
    for i, X in enumerate(datasets):
        # activate correct plot
        pyplot.subplot(3, 3, 3*model_type+i+1)

        # Get the correct optimized Theta for the current model_type and dataset
        current_theta = optimized_thetas_all_models[model_type][i]

        # Get the correct range for the current dataset
        current_range = ranges[i]

        # Set a title for each subplot
        pyplot.title(f"Model N{model_type} on X{i}")

        # call the plot function with the correct dataset with their ranges, and current_theta
        plot(X, model_type, current_theta, current_range)

pyplot.tight_layout()
pyplot.show()
```

